

Lecture Summary: Access Modifiers & Encapsulation in Python

Introduction The lecture focuses on how to control access to data within a class. This is achieved through access modifiers, which determine whether a variable or method can be viewed or modified from outside the class.

1. Public Access Modifiers By default, all members (variables and methods) in a Python class are Public.

- **Accessibility:** They can be accessed from inside the class, by an object (instance) outside the class, and by subclasses.
- **Class Variables:** Public class-level variables can be accessed directly using the Class Name (e.g., `Library.total_books`) without even creating an object.

2. Protected Access Modifiers Protected members are intended to be used only within the class and its subclasses (Inheritance).

- **Naming Convention:** Indicated by a single underscore prefix (e.g., `_variable`).
- **Python's Approach:** Unlike some other languages (like Java), Python does not strictly "block" access to protected members from the outside. It serves as a warning/convention to other programmers that this data should not be touched directly.
- **Inheritance:** Subclasses can freely access and use protected members of their parent class.

3. Private Access Modifiers (Note: While your summary focuses on Public and Protected, Private is the third tier in this system).

- **Naming Convention:** Indicated by a double underscore prefix (e.g., `__variable`).
- **Purpose:** To prevent any access from outside the class, even by subclasses. Python uses "Name Mangling" to make it difficult (though not impossible) to access these from outside.

4. The Role of Abstraction The lecture introduces Abstraction as the concept of hiding complex internal details and showing only the necessary features of an object. Access modifiers are the primary tool used to achieve this "data hiding."

Key Takeaways

- **Conventions over Restrictions:** Python relies on the "Adults Consent" rule—it assumes programmers will follow the naming conventions (underscores) rather than forcing strict technical barriers.
- **Encapsulation:** By using these modifiers, you maintain the integrity of your data and ensure that objects are used only in the ways they were intended.
-